

Praktikum 6: Hashtabellen

Die vollständige Implementierung (**HashTableOpenAddressing**) finden Sie in `vorlesung/L07_hashtable/`. Alle Aufgaben bauen darauf auf — Sie implementieren Erweiterungen oder analysieren das vorhandene Verhalten. Nutzen Sie die in `analyze_hashtable.py` definierte Hash- und Sondierungsfunktion.

1. Implementieren Sie eine Klasse **HashTableChaining**, die Kollisionen durch Verkettung auflöst, analog zu **HashTableOpenAddressing**. Jeder Slot enthält eine Python-Liste der gespeicherten Schlüssel. Bieten Sie folgende Methoden an:
 - a) `insert(x)`: Fügt `x` ein. Bereits vorhandene Werte werden nicht doppelt eingefügt.
 - b) `search(x)`: Gibt `True` zurück, falls `x` enthalten ist, sonst `False`.
 - c) `delete(x)`: Entfernt `x` aus der Liste des zugehörigen Slots, falls vorhanden.
 - d) `__str__()`: Gibt alle Slots mit ihren Ketten aus (auch leere Slots).
 - e) `alpha()`: Belegungsfaktor n/m (n = Gesamtanzahl gespeicherter Elemente über alle Listen).
2. Überprüfen Sie Ihre Implementierung, indem Sie eine Hashtabelle der Größe 20 anlegen und `seq0.txt` einfügen. Nutzen Sie die bekannte Hashfunktion.
 - a) Welchen Belegungsfaktor hat die Tabelle nach dem Einfügen?
 - b) Löschen Sie Eintrag 52 und überprüfen Sie mit `__str__`, ob die Liste des Slots geleert wurde.
 - c) Fügen Sie anschließend erneut `seq0.txt` ein. Überschreitet der Belegungsfaktor dabei 1? Kann `insert` bei der Verkettung je `False` zurückgeben?
 - d) Erklären Sie: Warum wird bei der Verkettung kein `DELETED_MARK` benötigt?
3. Legen Sie eine **HashTableOpenAddressing** der Größe 20 an und fügen Sie `seq0.txt` ein. Nutzen Sie die bekannten Hash- und Sondierungsfunktionen.
 - a) Löschen Sie den Eintrag 52. Was gibt `__str__` für den betroffenen Slot aus?
 - b) Betrachten Sie den Quellcode von `search`. Warum wird die Suche bei `DELETED` fortgesetzt, aber bei `UNUSED` abgebrochen?
 - c) Angenommen, `delete` würde die Zelle auf `UNUSED` setzen: Welche Operation würde dadurch fehlerhaft? Konstruieren Sie ein konkretes Beispiel.
 - d) Fügen Sie nach dem Löschen von 52 den Wert 24 ein. Berechnen Sie manuell (mit `h` und `f` aus `analyze_hashtable.py`), wo 24 landet, und verifizieren Sie dies mit `__str__`.
4. Legen Sie eine **HashTableOpenAddressing** der Größe 90 an und fügen Sie alle 100 Werte aus `seq1.txt` ein.
 - a) Wie viele Werte werden trotz freier Slots abgewiesen? Warum versagt die Sondierungsfunktion `f` bei dieser Tabellengröße?
 - b) Wiederholen Sie den Versuch mit Größe 89. Können mehr Werte aufgenommen werden? Erklären Sie, warum eine Primzahl als Tabellengröße im Vorteil ist.
 - c) Wiederholen Sie den Versuch mit Ihrer **HashTableChaining** (Größe 20, `seq1.txt`). Wie viele Werte werden aufgenommen? Wie groß ist der Belegungsfaktor?
 - d) Die theoretische mittlere Sondierungsanzahl bei offener Adressierung beträgt $1 / (1 - \alpha)$. Berechnen Sie diesen Wert für $\alpha = 0,5$ und $\alpha = 0,9$. Bis zu welchem α ist offene Adressierung in der Praxis noch akzeptabel?

5. Messen Sie für **HashTableOpenAddressing** (mit f) und **HashTableChaining** (mit h) die Anzahl der Vergleiche (`ctx.comparisons`) beim Suchen aller eingefügten Werte. Befüllen Sie Hashtabellen der Größen [50, 100, 200, 500, 1000] bis zu einem Belegungsfaktor von ca. 0,7 mit zufälligen Werten (`Array.random`) und stellen Sie die Ergebnisse in einem Plot dar.
- a) Welche Strategie erfordert bei gleichem α mehr Vergleiche?
 - b) Erhöhen Sie den Belegungsfaktor auf 0,9. Wie verändert sich das Bild?
 - c) Nennen Sie je einen Vorteil der Verkettung und der offenen Adressierung hinsichtlich Cache-Verhalten, Speicherverbrauch oder Parallelisierbarkeit.

Musterlösung Praktikum 6: Hashtabellen

1 Hashtabelle mit Verkettung

1.1 Implementierung HashTableChaining

Die vollständige Implementierung befindet sich im Repository unter `praktika/06_hashtable/aufgabe1_chaining.py`.

Die Klasse verwendet pro Slot eine Python-Liste; der Konstruktor erwartet die Hashfunktion `h` als Parameter. Der Slot-Index ergibt sich aus `int(h(x, m))`. Alle Vergleiche (`elem == x`) laufen über `Int`-Objekte und werden vom `AlgoContext` gezählt. Der Belegungsfaktor $\alpha = n/m$ kann bei der Verkettung – anders als bei offener Adressierung – den Wert 1 überschreiten.

2 Test mit `seq0.txt` ($m = 20$)

2.1 a) Belegungsfaktor nach dem Einfügen

`seq0.txt` enthält 14 Werte. Nach dem Einfügen gilt:

$$\alpha = \frac{n}{m} = \frac{14}{20} = 0,70$$

Die Tabelle nach dem Einfügen (Auszug belegter Slots):

```
[ 0] 34 -> 47
[ 2] 52
[ 5] 15
[ 8] 46
[ 9] -59
[10] 51
[11] -77
[13] 14 -> 48
[15] -87
[16] 58
[18] -50 -> 50
```

Slots 0, 13 und 18 zeigen Kollisionen mit je zwei Elementen.

2.2 b) Löschen von Eintrag 52

```
ht.delete(Int(52, ctx))
```

Slot 2 ist anschließend leer (--); der Belegungsfaktor sinkt auf $\alpha = 13/20 = 0,65$.

2.3 c) Erneutes Einfügen von `seq0.txt`

Da `insert` Duplikate ablehnt, wird nur der zuvor gelöschte Wert 52 neu eingefügt; alle anderen Werte sind noch vorhanden. Der Belegungsfaktor kehrt auf 0,70 zurück.

Kann $\alpha > 1$ werden?

Ja – bei der Verkettung können beliebig viele Elemente gespeichert werden, da die Ketten

unbegrenzt wachsen. Werden ausschließlich neue (nicht bereits enthaltene) Werte eingefügt, übersteigt α die Grenze 1. Im vorliegenden Fall verhindert die Duplikatprüfung dies.

Kann insert je False zurückgeben?

Nicht wegen “Tabelle voll” – die Tabelle hat keine Kapazitätsgrenze. In der obigen Implementierung gibt `insert` `False` zurück, wenn der Wert bereits enthalten ist (Duplikatschutz). Eine Variante ohne Duplikatschutz würde immer `True` zurückgeben.

2.4 d) Warum kein DELETED?

Bei offener Adressierung markiert `DELETED` einen gelöschten Slot, damit `search` die Sondierungssequenz nicht vorzeitig abbricht. Bei der Verkettung gibt es keine Sondierungssequenz: `search` durchläuft vollständig die Liste eines einzelnen Slots. Ein gelöschtes Element wird einfach aus der Liste entfernt – kein Platzhalter wird benötigt, und nachfolgende Suchen in derselben Liste bleiben unberührt.

3 Löschen bei offener Adressierung

3.1 a) delete(52) – was steht im Slot?

```
ht = HashTableOpenAddressing(20, f, ctx)
# seq0.txt einfuegen, dann:
ht.delete(Int(52, ctx))
```

Der Slot $h(52) = 2$ enthält danach den Wert `DELETED`. `__str__` gibt aus:

```
[34, UNUSED, DELETED, UNUSED, 50, 15, 47, UNUSED, 46, -59,
 51, -77, UNUSED, 14, UNUSED, -87, 58, UNUSED, -50, 48]
```

3.2 b) Warum search bei DELETED weitermacht

`search` prüft in jeder Iteration:

- `UNUSED` → Abbruch: Der gesuchte Wert wurde *nie* über diesen Slot hinaus sondiert, er ist nicht vorhanden.
- `DELETED` → weitersuchen: Der Wert könnte durch eine frühere Kollision über diesen Slot hinaus platziert worden sein. Dieser Eintrag *erhält* die Sondierungssequenz aufrecht.
- Gefundener Wert → Treffer.

3.3 c) Konkretes Gegenbeispiel: UNUSED statt DELETED beim Löschen

Die Werte `-50` und `50` haben beide $h = 18$ und damit dieselbe Sondierungssequenz:

Wert	h	$f(\cdot, 0)$	$f(\cdot, 1)$	$f(\cdot, 2)$
-50	18	18	4	0
50	18	18	4	0

Szenario: `-50` wird zuerst eingefügt und belegt Slot 18. `50` kollidiert dort und landet daher in Slot $f(50, 1, 20) = 4$. Wird nun `-50` gelöscht und Slot 18 auf `UNUSED` gesetzt, bricht `search(50)` bereits bei Slot 18 ab – obwohl `50` in Slot 4 liegt. Das Ergebnis wäre falsch negativ. Mit `DELETED` läuft die Suche weiter: Slot 18 (`DELETED`) wird übersprungen, Slot 4 wird geprüft, `50` wird gefunden.

3.4 d) Wo landet 24 nach delete(52)?

Berechnung der Sondierungssequenz für Schlüssel 24 (Tabellengröße $m = 20$, Goldener Schnitt $A \approx 0,6180$):

$$h(24) = \lfloor \{24 \cdot A\} \cdot 20 \rfloor = \lfloor 0,833 \cdot 20 \rfloor = 16$$

Schritt i	$f(24, i, 20)$	Inhalt
0	16	belegt (58)
1	2	DELETED → freier Slot

24 landet in **Slot 2** (dem früheren Slot von 52). **insert** belegt DELETED-Slots, da diese als frei gelten.

4 Belegungsfaktor und Kapazitätsgrenzen

4.1 a) HashTableOpenAddressing, $m = 90$, seq1.txt

Tabelle	Eingefügt	Abgewiesen	Freie Slots
$m = 90$	87	13	3

Obwohl noch 3 Slots frei sind, werden 13 Werte abgewiesen. Ursache: Die Sondierungsfunktion $f(x, i) = (h(x) + i + 5i^2) \bmod m$ erzeugt für $m = 90$ (keine Primzahl) zyklische Sequenzen, die nicht alle Slots abdecken. Die 3 verbleibenden Slots liegen außerhalb jeder erreichbaren Sondierungssequenz.

4.2 b) HashTableOpenAddressing, $m = 89$ (Primzahl)

Tabelle	Eingefügt	Abgewiesen	Freie Slots
$m = 89$	89	11	0

Alle 89 Slots werden vollständig genutzt; die 11 Abweisungen entstehen, weil die Tabelle physisch voll ist ($n = 89$, $m = 89$). Bei Primzahlgröße erreicht die quadratische Sondierung alle m Slots, sodass kein Slot ungenutzt bleibt.

4.3 c) HashTableChaining, $m = 20$, seq1.txt

seq1.txt enthält 100 Werte, davon 77 eindeutige. Da **insert** keine Duplikate speichert, werden genau 77 Werte aufgenommen:

$$\alpha = \frac{77}{20} = 3,85$$

Die Verkettung hat keine Kapazitätsgrenze: Alle eindeutigen Werte werden stets aufgenommen, unabhängig von α .

4.4 d) Theoretische Sondierungsanzahl

Die mittlere Anzahl der Sondierungen bei erfolgreicher Suche beträgt $\frac{1}{1-\alpha}$:

α	$\frac{1}{1-\alpha}$
0,5	2,0
0,9	10,0

Einen nicht-willkürlichen Grenzwert liefert der Vergleich mit der Verkettung, deren mittlere Suchkomplexität die Vorlesung als $O(1 + \alpha)$ angibt. Setzt man die Kosten der offenen Adressierung gleich dem Doppelten:

$$\frac{1}{1 - \alpha} = 2(1 + \alpha) \implies 1 = 2(1 - \alpha^2) \implies \alpha = \frac{1}{\sqrt{2}} \approx 0,71$$

Ab $\alpha = 1/\sqrt{2}$ benötigt die offene Adressierung mehr als doppelt so viele Vergleiche wie Verkettung. Die gängige Praxisgrenze $\alpha < 0,75$ orientiert sich an diesem theoretischen Schwellenwert.

5 Empirischer Vergleich

5.1 Messergebnisse

Vergleiche beim Suchen aller eingefügten Werte für beide Strategien (zufällige Werte, je Tabellengröße m):

m	OA ($\alpha \approx 0,7$)	Chaining ($\alpha \approx 0,7$)	OA ($\alpha \approx 0,9$)	Chaining ($\alpha \approx 0,9$)
50	146	49	220	61
100	254	94	424	125
200	474	191	858	252
500	1 300	467	2 254	604
1 000	2 482	901	4 654	1 256

5.2 a) Welche Strategie benötigt mehr Vergleiche?

Die offene Adressierung benötigt bei gleichem α stets deutlich mehr Vergleiche. Kollisionen erzeugen lange Sondierungssequenzen: Jeder besuchte Slot kostet einen Vergleich, auch wenn er den gesuchten Wert nicht enthält. Bei der Verkettung ist nur die jeweilige Kette zu durchsuchen – im Schnitt $\alpha/2$ Vergleiche pro erfolgreicher Suche.

5.3 b) $\alpha = 0,9$

Der Nachteil der offenen Adressierung wird drastisch: Die Vergleichszahl verdoppelt sich gegenüber $\alpha = 0,7$, was der Formel $1/(1 - 0,9) = 10$ Sondierungen entspricht. Bei der Verkettung wächst die mittlere Kettenlänge nur auf $\alpha = 0,9$, also knapp eine zusätzliche Position. Die Schere zwischen beiden Strategien öffnet sich mit steigendem α erheblich.

5.4 c) Systemspezifische Vor- und Nachteile

- **Verkettung:** Einfaches Löschen (kein DELETED-Eintrag nötig); keine Kapazitätsgrenze; gute Parallelisierbarkeit (separate Ketten können unabhängig gesperrt werden).
- **Offene Adressierung:** Besseres Cache-Verhalten (alle Einträge im zusammenhängenden Array, keine Pointer-Indirektion); geringerer Speicherverbrauch (kein Listenknoten-Overhead).